

The App Development Standard

A portable engineering, security, and process bible for every app.

STATUS: live — the reusable, cross-app standard for building complete, hardened, international-quality applications from day one. It distills the architecture, security posture, quality gates, and operating process that were proven and hardened on a production matrimony platform (web + admin + mobile + backend) across dozens of full audits, and generalizes them so the **same rules, the same stack, and the same guarantees** apply to any new app — business, consumer, or internal — regardless of its front-end feature set.

How to use this document

This is a **meta-standard**: it defines *how* an app is built, secured, tested, and shipped — not *what* the app does. Copy it into every new repository (`docs/standards/APP_DEVELOPMENT_STANDARD.md`) at Sprint 0 so the rules travel with the code and every cloud/remote session that clones the repo obeys them.

- **When starting a new app:** work the Day-One Setup Checklist top to bottom. It stands up the stack, the CI gates, the security baseline, and the prevention system *before* the first feature.
- **When adding a feature:** run the Blast Radius Gate → build to the module's depth → verify against the Definition of Done.
- **When reviewing whether something is "done":** it is done only when it passes the Definition of Done — not when "it renders" or "the happy path works."

Two words are used precisely throughout:

- **MUST** — a hard requirement. A change that violates it is not done, in the same way a crash is not done. Wherever possible, MUSTs are enforced by CI, not by human review.
- **SHOULD** — a strong default. Deviating is allowed only with a documented reason recorded in the repo.

The governing idea: **make correctness structural**. Every rule below that can be enforced by a machine is enforced by a machine, so quality does not depend on anyone remembering it.

-
1. First principles
 2. The standard technology stack
 3. Repository & architecture
 4. The security & safety baseline (mandatory in every app)
 5. The enforced-seams prevention system
 6. Data, privacy & compliance
 7. Reliability, performance & scale
 8. Accessibility
 9. Internationalization
 10. Observability & operations
 11. Testing strategy

12. CI/CD and the merge gate
 13. Deployment & infrastructure
 14. The Blast Radius Diagnosis Gate
 15. The build process & the owner model
 16. Admin & owner tooling (including the Admin Control MCP)
 17. The Definition of Done
 18. Day-One setup checklist (Sprint 0)
 19. Forbidden anti-patterns
-

These are the non-negotiable convictions behind every rule that follows. If a decision is ever unclear, decide in the direction that best serves these.

1.1 Anti-skeleton: complete, not "it renders"

The most common failure is a **skeleton** — an app that looks done but is hollow underneath. A feature is **done only when it is complete**, never when "it renders" or "the happy path works." For every entity and module hold this bar:

- **Complete lifecycle** — create, read-one, read-list (with search / filter / sort / pagination), update, delete/archive, state transitions, relationships, history, bulk actions, export. Never just "create and list."
- **Every state handled** — loading (skeletons, no layout shift), empty (guiding, not blank), error (clear + recovery, never a raw error or endless spinner), partial, success, offline, unauthorized, not-found. No white-screen crash; no stale UI after a write.
- **Every list and form robust** — server-side search/filter/sort/paginate; inline + server validation; double-submit prevention; input preserved on failure; undo on destructive actions.

1.2 Three audiences, always

Every feature serves **three** audiences, not one:

1. **The end user** who uses it.
2. **The staff/admin roles** who manage, moderate, support, and reconcile it.
3. **The owner** who oversees and controls it, with drill-down visibility into every number.

A feature built only for the end user is **one-third done**. Design the admin and owner surface in the same change as the user surface.

1.3 Prevention over patching

Bugs are not fixed one call-site at a time — **bug classes are made extinct**. Every cross-cutting invariant (money, session-minting, authorization, quotas, redaction, idempotency) lives as **one shared primitive**, and a CI architecture gate makes bypassing it a build failure. Every new finding is closed by **adding or tightening a gate in the same change as the fix**, never a one-off spot patch. The set of "can never happen again" classes only ever grows. This is Section 5 — the single most important durable-quality mechanism in this standard.

1.4 Fail closed on anything that matters

Safety, money, entitlement, and authorization gates **deny on a missing or ambiguous signal**. Unknown feature flag → off. Missing webhook secret → reject. Provider can't be verified → don't grant. Store/limiter down → refuse the privileged action. The rare, *documented* fail-open exceptions exist only where denying on a transient signal would harm legitimate users **and** an independent hard guard already sits behind it (see 4.9).

1.5 The owner is non-technical

Assume the owner does not read code and never will. The builder owns **everything technical** — building, reviewing, verifying, securing, fixing — taking the whole app into account on every change. The owner owns **product** — UX, UI, wording, pricing, what "done" looks like. **Never hand the owner a technical task** (no code review, no commands to run, no logs to interpret). Any process that would create a recurring human review/decision task is mis-scoped and must be re-shaped into an agent-executed + machine-enforced task. Proof of work is working screens and preview links, never diffs or logs.

1.6 Portability: own your data, rent only what you must

Choose the stack so that **switching any vendor later is a config change, not a rewrite** (Section 2.4). Data lives in open, portable stores (standard Postgres, S3-API object storage). Every external channel (email, SMS, payments, push, AI) sits behind a provider interface with a dev/log implementation, so a leaked or deprecated vendor key is swapped by writing one file and changing one config line.

1.7 The single test for "is it done"

Could a lean team run a real business on this the day it ships — with real paying customers — without a backlog of missing operations blocking them? If not, it is not done.

One backend, one typed client, shared types — **true multiplatform from a single codebase**. This is the reference stack, proven in production. Adapt only with a documented reason, and never at the expense of the safeguards in Sections 4–5.

The "current" discipline. "Current" means *verify the latest stable at project start and pin it in the repo*. Never install what an old document remembers. Pin exact ranges in the manifests, keep one lockfile, and let a dependency bot (see 12.7) propose upgrades. The versions below are the proven baseline at the time of writing; treat them as the floor, re-verify at Sprint 0, and record what you pinned.

2.1 Foundation & tooling

Concern	Choice	Baseline version	Notes
Language	TypeScript (strict)	<code>^5.6</code>	One shared version across the workspace; <code>strict</code> + <code>noUncheckedIndexedAccess</code> ,

Concern	Choice	Baseline version	Notes
			<code>noUnusedLocals/Parameters</code> , <code>noImplicitReturns</code> , <code>isolatedModules</code> .
Runtime	Node.js	<code>>=20</code> (LTS)	Pin in <code>engines</code> ; Docker builds on <code>node:20-alpine</code> .
Package manager	pnpm	<code>9.x</code> pinned via <code>packageManager</code>	Frozen-lockfile installs in CI and Docker.
Monorepo orchestrator	Turborepo	<code>^2.3</code>	Tasks: <code>build</code> , <code>dev</code> , <code>lint</code> , <code>typecheck</code> , <code>test</code> , <code>test:e2e</code> .
Formatting	Prettier	<code>^3.3</code>	<code>singleQuote</code> , <code>trailingComma: all</code> , <code>printWidth: 100</code> , LF endings.
Linting	ESLint flat config + <code>typescript-eslint</code>	<code>9.x</code> / <code>8.x</code>	Type-aware. Carries the architecture-enforcement lint rules (5.5).
Editor baseline	<code>.editorconfig</code>	—	UTF-8, LF, 2-space, final newline, trim trailing whitespace.

2.2 Backend

Concern	Choice	Baseline	Notes
Framework	NestJS	<code>11.x</code>	Modular DI; one module per domain.
Database	PostgreSQL	<code>16</code>	The only durable store that <i>must</i> be protected. Extensions kept portable: <code>uuid-oss</code> , <code>pgcrypto</code> , <code>pg_trgm</code> , <code>citext</code> (+ <code>pgvector</code> where embeddings are used).
ORM	Prisma	<code>7.x</code>	New TS client generator (no Rust engine); driver adapter over <code>pg</code> . Migrations additive & forward-only .
Cache / queue	Redis + BullMQ	<code>ioredis</code> <code>5.x</code> / <code>bullmq</code> <code>5.x</code>	Cache, throttle store, background jobs. Disposable — fail open / fall back.
Realtime	Socket.IO	<code>4.x</code>	Chat, presence, typing, call signalling; Redis adapter for multi-instance.
Search	Meilisearch	<code>v1.10</code>	Postgres full-text fallback if it is down. Private networking only.
Auth/crypto	<code>argon2</code> (id), <code>passport</code> , <code>passport-jwt</code> , <code>otplib</code> , <code>@nestjs/jwt</code>	—	See 4.2.
Object storage	S3-API SDK (<code>@aws-sdk/client-s3</code>)	<code>3.x</code>	Private buckets, signed URLs only. R2 today, any S3 provider tomorrow.
Validation	<code>class-validator</code> + <code>class-transformer</code> + <code>zod</code>	—	DTO whitelisting on the wire; Zod for env + shared schemas.
Observability	OpenTelemetry, Prometheus (<code>prom-client</code>), <code>pino</code> + <code>pino-http</code> , Sentry	OTel <code>2.x</code>	See Section 10.

Concern	Choice	Baseline	Notes
Security	<code>helmet</code>	<code>8.x</code>	CSP, HSTS, frame/ <code>nosniff</code> /referrer headers.
Test runner	Vitest	<code>4.x</code>	Unit + coverage gate; e2e boots the real app (Section 11).
Deploy	Docker (multi-stage <code>node:20-alpine</code>) on Fly.io	—	<code>prisma migrate deploy</code> as the release command.

2.3 Web, admin & mobile

Surface	Choice	Baseline	Notes
Web (end users)	Next.js App Router, installable PWA (Serwist)	Next <code>16.x</code> , React <code>19.x</code> , Serwist <code>9.x</code>	Bilingual+ routing, service worker + offline.
Admin (staff/owner)	Next.js App Router, PWA	same as web	Separate app, separate deploy; charts via Recharts.
Mobile (end users)	Expo + React Native, New Architecture	Expo SDK <code>54</code> , RN <code>0.81</code> , <code>expo-router</code> <code>6.x</code>	iOS + Android from one codebase; NativeWind for styling.
Styling	Tailwind CSS (web/admin) · NativeWind (mobile)	<code>3.4</code>	Shared design tokens through <code>ui-core</code> .
i18n	<code>next-intl</code> (web/admin) · <code>i18next</code> (mobile)	—	One catalogue feeds both (Section 9).
Web/admin tests	Vitest + Testing Library + <code>axe-core</code> ; Playwright e2e (with <code>@axe-core/playwright</code>)	—	a11y assertions in-suite.
Mobile tests	Jest (<code>jest-expo</code>) + Testing Library; Detox e2e; Maestro smoke	—	Serial run in CI; snapshot discipline (11.5).
Monitoring	Sentry across all platforms	<code>@sentry/*</code> <code>10.x</code> (<code>7.x</code> RN)	Error capture; backend adds OTel tracing + metrics.

2.4 The zero-lock-in rule

Portability is a first-class requirement, achieved by architecture rather than wishful thinking:

- **Database** — standard Postgres. Dump-and-restore moves you to any provider (RDS, Supabase, Crunchy, self-hosted) with zero code change. Use no proprietary extensions beyond the portable set above.
- **Object storage** — the S3 API. R2 → S3 → B2 → MinIO is a config change.
- **Auth** — self-hosted; you own the tables and the argon2 hashes. No auth-as-a-service lock-in, no PII residency question.
- **Push** — the one unavoidable platform tie is Android FCM (the OS won't deliver otherwise). Minimize it: call the **FCM HTTP v1 API directly** (no `firebase-admin` SDK) and talk to **APNs directly** for iOS; Web Push uses self-VAPID. Everything sits behind a `PushProvider` interface, so swapping is a ~100-line file.

- **Every external channel** (email, SMS, payments, AI, translate, moderation) is behind a **provider registry** with a circuit breaker and a dev/log provider. Keys are isolated behind the interface; a leaked vendor key is swappable without touching business logic.

2.5 Shared packages (the contract layer)

Extract the cross-surface contracts into workspace packages so web, admin, and mobile share one source of truth and the backend cannot drift from its clients:

Package	Purpose
shared-types	TypeScript types shared across clients.
validation	Zod schemas reused on client and server.
api-client	Typed client generated from the backend's OpenAPI contract — the wire is never hand-typed.
ui-core	Headless, platform-agnostic UI logic (design tokens, class maps, a11y contract builders) — no JSX.
ui-components	Shared React DOM components for web + admin.
i18n	The translation catalogue + locale/RTL registry.

Backend build boundary (a real, subtle trap). If the backend's `tsconfig` `paths-maps` `@app/*` to package `source`, then importing a workspace package from backend `src/` pulls `packages/*/src` into the build, moves its `rootDir`, and emits the entrypoint to the wrong path — crash-looping the container. **Define backend types locally; never import the shared packages from backend `src/`.** Add a post-build assertion that the compiled entrypoint lands at the expected path (12.2).

3.1 One repo per app, fully isolated

Each app gets **its own repository, its own uniquely-named Docker project** (its own container names, networks, volumes, ports), and its own cloud resources. Apps never collide and never share mutable state. This is both an operational rule and a safety rule (see 15.4).

3.2 Monorepo layout

apps/	
backend/	one API for every client (REST + WebSockets), versioned /v1
web/	end-user Next.js PWA
admin/	staff + owner Next.js PWA
mobile/	Expo React Native (iOS + Android)
admin-mcp/	MCP server exposing the whole admin surface (Section 16)
packages/	shared-types · validation · api-client · ui-core · ui-components · i18n · domain
infra/	docker-compose for local services, observability stack, load tests, DR scripts
docs/	architecture (ADRs), standards, security, runbooks, the generated API contract

All clients talk to the **single backend API**. There is no per-client backend. The API is versioned (`/v1/...`) and self-documents via **OpenAPI**, which is generated from the code and stale-checked in CI so the committed contract always matches reality.

3.3 The backend is a set of domain modules

Structure the backend as many small, self-contained domain modules (`*.module.ts` + service(s) + controller(s)

- DTOs + specs), wired in one root module. Group them by domain: platform/cross-cutting (config, prisma, redis, cache, crypto, observability, audit, feature-flags, health, storage), identity & safety (auth, users, RBAC, verifications, moderation, risk/trust-&-safety, reports/blocks), the product domains, messaging, monetization & finance, and compliance/ops (privacy, analytics, staff, support). Cross-cutting invariants do **not** live inside these modules — they live in shared primitives the modules route through (Section 5).

3.4 The request pipeline (defense in depth)

Every request passes an ordered chain of cross-cutting guards, so security and correlation are structural, not per-handler:

```
client (web / admin / mobile)
  | HTTPS, Bearer JWT (or HttpOnly cookie for staff), x-request-id
  ▼
[ Request-ID middleware ] binds correlation id (requestId + traceId) into async-local storage
  | PRE-GUARD, so even a 429 log carries the id
  ▼
[ Hardened rate-limit guard ] per-user/IP; 429 = structured body + Retry-After + metric + cor
  ▼
[ Auth guard / Staff guard ] authenticates; re-checks account status; enforces RBAC + MFA
  ▼
[ Validation pipe ] whitelist + reject-unknown + transform DTOs
  ▼
[ Request-logging interceptor ] RED metrics + access log, augmenting the correlation context
  ▼
[ Controller → Service ] business logic → Postgres / Redis / search / object storage / p
  ▼
[ Exception filter ] maps errors, echoes x-request-id, sends only 5xx to error track
  ▼
response (+ x-request-id echoed)
```

Real-time (chat/presence/calls) is a **second authenticated surface** — the WebSocket handshake verifies the token and is rate-limited **before** any token verify or DB lookup. Background jobs run on a Redis-backed queue.

This is the concrete answer to "list all the basic security features that must be inside any app I build." Each item below is a **MUST** for every app — consumer-facing and admin — regardless of feature set. Each is stated as a requirement plus the proven reference mechanism so it can be implemented identically everywhere.

4.1 Audit logging — tamper-evident, append-only

MUST: every meaningful privileged, financial, safety, or account action is recorded in an **append-only, tamper-evident audit log** that is the system of record for "who did what, when, from where."

Reference mechanism:

- One central `AuditService.log(event)`; nothing writes audit rows directly.
- Each row stores `prevHash` and `hash`, where `hash = SHA-256(prevHash + "|" + canonicalPayload)` — a **hash chain**, so editing any historical row invalidates every row after it.
- Writes are serialized with a **transaction-scoped advisory lock** on a single constant key, so concurrent and multi-instance writers still form one linear chain. The chain tail is found *by link* (the row no other row references as its predecessor), not by timestamp.
- The `metadata` (JSONB) is **canonicalized** (keys sorted at every depth) before hashing so a JSON round-trip never changes the hash.
- A `verifyChain()` routine walks genesis → tail and reports forks, hash mismatches, and dangling links.
- A `logInTx(tx, event)` variant appends **inside** a caller's transaction, so an audit row commits or rolls back atomically with the action it records (e.g. session revocation on password reset).
- Each row carries `category`, `action`, `actorUserId / actorStaffId`, `targetType / targetId`, `ip`, `userAgent`, `metadata`, and **impersonation attribution** filled centrally from the request context (so every "owner-acting-as-member" or "staff-acting-as-member" action is correctly attributed for free).
- When an action is driven by an automated operator (e.g. the Admin MCP), forgeable operator headers are folded into metadata **explicitly marked UNVERIFIED**; the non-repudiable identity remains the authenticated actor.

4.2 Authentication

MUST: self-hosted, standards-based auth with short-lived access tokens, rotating refresh tokens with reuse detection, memory-hard password hashing, and MFA for privileged accounts.

Reference mechanism:

- **Access token** — JWT, **algorithm pinned** (e.g. `HS256`; never accept `alg:none` or an unexpected algorithm), short TTL (**15 min**), carrying `sub`, `type`, tier, and a session id as `jti`.
- **Refresh token** — opaque `sessionId.secret`; only its **SHA-256 hash** is stored; compared constant-time. Two independent caps: a **sliding idle TTL** (re-armed on each rotation, e.g. 90 days) and an **absolute hard cap** from origin login (e.g. 180 days) that an always-refreshing client still cannot exceed.
- **Rotation** happens in one **serializable** transaction (retried on serialization conflicts). **Refresh-reuse detection:** presenting an already-rotated token whose successor was used → hard 401 and **revoke the whole token family**. A wrong secret on a live session → immediate revoke.
- **Passwords** — **argon2id** (memory ≈ 64 MB, time cost 3, parallelism 4; tune to hardware). Minimum length 10. Check new passwords against a breached-password corpus. No forced periodic rotation (modern guidance). Anti-enumeration: burn equal hashing time on unknown identities; return one generic "invalid credentials" for every failure; one generic "account already exists" that never reveals which identifier collided.
- **OTP (phone/email)** — 6 digits, hashed at rest, short expiry, **atomic attempt burn** (so parallel requests can't brute-force), **purpose binding** (a login code cannot satisfy a password reset), single-use claim,

and the code is **never** returned in an API response. Rate + cost caps per identity, per IP, and a global daily kill-switch.

- **Sessions** — every login creates a session row (device, IP, user-agent, last-used). Users can list and revoke sessions. Password change/reset revokes **every** session and drops live sockets.
- **Account-status gate** — every token-minting rail calls a shared `assertAccountActive` check; banned / suspended / deleted / legal-hold accounts cannot mint or refresh a token, and the auth guard re-checks status on **every** request (a suspension takes effect immediately, not at next login).
- **Staff/owner MFA** — TOTP mandatory for owner/super-admin and for anyone holding a sensitive permission; a session that hasn't satisfied MFA is 403'd on privileged routes; enrollment routes are explicitly exempt.

4.3 Authorization & RBAC

MUST: all authorization is server-side and role-based; the client's claim of its own tier, role, or price is never trusted.

Reference mechanism:

- A **staff guard** reads the token (preferring an **HttpOnly cookie** for the admin console, Bearer as back-compat), then checks a `@RequireStaffPermission(...slugs)` decorator against the account's effective permissions (role grants ± per-account overrides, with **expired overrides ignored** in both directions).
- Exactly **one** role may bypass per-slug checks (`owner`); it never bypasses authentication or the MFA gate, and when the bypass carries the owner past a **finance-scoped** permission they don't individually hold, a **break-glass audit row** is written automatically.
- **401 for unauthenticated, 403 for authenticated-but-unauthorized** — kept distinct.
- **Entitlements** (paid tiers/features) are enforced by a **shared tier-gate** reading server-side entitlement state; the client's tier is never trusted. Feature flags are **server-enforced** and **fail off**.
- **Impersonation / "view as"** is **read-only by construction**: the view-as token is non-refreshable and a global interceptor rejects every non-GET/HEAD/OPTIONS request made with it. A separate, tightly-scoped, owner-only, fully-audited "full control" mode (if it exists) is undiscoverable to non-owners (generic 404) and identified only by a server-side secret.
- Every permission slug referenced in code **must exist in the seed** — enforced by a build gate, so there are no phantom (silently fail-closed-dead) permissions.

4.4 Rate limiting & abuse control

MUST: a global rate limiter on every route, keyed so an authenticated user cannot dodge it, plus tighter per-route caps on sensitive endpoints and a per-identity login logout.

Reference mechanism:

- A **hardened throttler guard** registered globally. The tracking key is the **authenticated user id** when present (so rotating source ports doesn't help), else the IP — and the framework's `trust proxy` is set so a spoofed `X-Forwarded-For` can't move the IP.
- A **429** returns `Retry-After`, `X-RateLimit-*` headers, and a **parseable JSON body** (`{ code: 'rate_limited', retryAfterSeconds }`), increments a labelled rejection metric, and emits a correlated WARN.

- Tiered buckets (per-second burst, per-minute, per-hour) plus a **hard global abuse bucket no route override can relax**. ~Sensitive routes (login, password reset, OTP, reports, search) get explicit tight overrides.
- **Login lockout** keyed by **identity** (tight threshold) with a **coarse per-IP backstop** (to avoid CGNAT/shared-IP collateral lockout), checked before the expensive password verify.
- **Slowloris/DoS** hardening: request/keep-alive/headers timeouts, a small body-size cap (e.g. 1 MB), and bounded timeouts + redirects on all outbound HTTP.

4.5 Secrets management

MUST: secrets live in the platform's secret store, never in the repo; the app validates its entire environment at boot and **refuses to start** on an invalid or insecure-default configuration in production.

Reference mechanism:

- One **Zod env schema** validated at boot by a config service; an invalid env throws and the app won't boot. All access is typed (`config.get(key)`) — **no raw** `process.env` in feature code.
- Booleans are mapped **explicitly** (the string `"false"` must never enable a flag) — avoid `z.coerce.boolean` footguns, especially for any `*_TEST_MODE` / sandbox flag.
- A list of **insecure-default secrets** is checked in production: any still at its dev default emits a CRITICAL log + error-tracker capture, and a `STRICT_SECRETS=true` switch promotes that to a **hard boot failure**.
- **Test-mode / sandbox flags in production** → **refuse to boot** (they bypass OTP delivery or capture real funds). A build gate enumerates every `*_TEST_MODE` flag in the code and fails CI if one isn't covered by the boot guard.
- **Secret scanning** on every PR (see 12.5); an allowlist covers only known CI test values.
- **Distinct secrets rotate independently** — the JWT signing secret, the encryption key, the blind-index key, and any hashing salt are separate values.

4.6 PII protection & encryption at rest

MUST: sensitive personal data is minimized in responses, encrypted at rest where the threat model warrants, searchable only via keyed blind indexes, and never written to logs.

Reference mechanism:

- **Field-level encryption** for the most sensitive identifiers: **AES-256-GCM** with a fresh random IV per value, stored as a self-describing packed buffer `[IV][tag][ciphertext]`, so the same plaintext never yields the same bytes.
- **Blind indexes** for equality lookups on encrypted fields: **HMAC-SHA256** of the *normalized* value under a **separate** key — deterministic (so it can carry a unique constraint) but keyed (so the small email/phone space can't be rainbow-tabled).
- Crypto helpers **fail soft** so a key/env problem never breaks signup or login; a decrypt-on-read resolver routes all display/send/export sites through one function, and ciphertext is stripped from every response.
- **Government-ID numbers** (NID/passport/licence) are stored **only as a salted hash** for the "one document = one account" uniqueness check — the raw number is never persisted.
- **Global secret-column stripping** at the ORM layer: password hashes, MFA secrets, and refresh-token hashes are omitted from every read by default; the handful of legitimate readers opt back in per-query.

- **Log redaction**: exact-path redaction plus a recursive scrubber that redacts by key-name fragment (`password` , `secret` , `token` , `otp` , `nid` , `phone` , `email` , `dob` , ...), asserted by test so PII can't reach the logs.

4.7 Input validation

MUST: every request body is a typed, whitelisted DTO; unknown properties are rejected; text is normalized to defeat evasion.

Reference mechanism:

- A **global validation pipe** with `whitelist: true` , `forbidNonWhitelisted: true` , `transform: true` — unknown properties are stripped **and rejected**, and bodies transform to their DTO types.
- A build gate **bans inline-object** `@Body()` **types** (which would bypass validation) — bodies must be DTO classes with validators.
- Shared **text normalization** (NFKC + strip bidirectional/zero-width characters + fold confusable digits) on free-text inputs, to defeat Unicode-evasion of moderation and contact-info filters.
- Pagination inputs are clamped (no 500 on a malformed `limit`); numeric/date inputs are range-checked.

4.8 Trust, safety & content moderation

MUST: any app with user-to-user contact or content has a trust-&-safety layer: reporting, blocking, moderation, and a risk-signal pipeline that **surfaces to staff and never silently enforces**.

Reference mechanism:

- **Risk engine** — behavioral bot/fake-account/underage-signal detection that aggregates into a score and creates a **pending staff-review item**; it **never auto-bans**. Fingerprints/IPs/UAs are stored only as domain-salted hashes, never raw, and risk signals are staff-only.
- **Abuse-specific classifiers** (e.g. scam/fraud rules) scan message bodies **in memory, never persisted** — only PII-free match metadata is stored. Recipient warnings are **generic** and never reveal which rule fired.
- **Reports & blocks** — a block tears down the live socket room **and** blocks reading history (entitlement is re-evaluated on teardown, not only at initiate — see 5.2 seam 7).
- **SOS/safety** signals are **always** recorded, audited, and queued regardless of any feature flag; any "notify my trusted contact" flow requires **confirmed, consented** contacts (one-time code, hashed, constant-time compare, attempt-capped).
- **Identity verification** (selfie liveness, document face-match) **fails closed to a manual staff queue** on any error or uncertainty — it never auto-approves.

4.9 Fail-closed is the default

MUST: money, authorization, entitlement, and safety decisions deny on a missing/ambiguous signal.

Examples that are all fail-closed by construction: unknown feature flag → off; payment amount mismatch (collected vs booked) → refuse to grant + audit; unverifiable in-app-purchase receipt → reject; `/metrics` with no auth token in prod → 403; missing webhook secret → reject; banned/suspended account → cannot mint a token; consent binds to the **server's** current policy version; under-18 DOB → hard block.

The only fail-**open** exceptions are explicitly documented, chosen where denying on a transient signal would harm legitimate users, and each has an independent hard guard behind it (e.g. an OTP global-cap DB blip doesn't block all logins because the per-identity cap and throttler still apply; a logging hiccup never blocks an authz decision).

4.10 Transport, headers, CORS, webhooks & payment integrity

MUST:

- **HTTPS everywhere**, forced at the edge. **HSTS** with a long max-age + `includeSubDomains` + `preload`.
 - **Helmet-class security headers**: a restrictive **CSP** (`default-src 'self'`, `object-src 'none'`, `frame-ancestors 'none'`, `base-uri 'self'`), `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: no-referrer`, and remove `X-Powered-By`.
 - **CORS** from an explicit allow-list (never `*` with credentials).
 - **API docs (Swagger/OpenAPI UI) gated out of production** unless explicitly opted in — no free recon.
 - **Webhook signature verification** on every inbound provider callback (verify against the **raw** body; reject on a missing secret/signature). Never trust a provider's redirect status — **verify server-to-server** against the provider's API before granting anything.
 - **Idempotency** everywhere money or grants are involved: client-supplied idempotency keys are **never** used raw as a spend key — they are namespaced under a server-derived scope against a unique constraint; webhook handlers dedupe on the external event id and use an advisory lock + status re-read to guarantee exactly-once.
 - **Object storage** is private; reads and writes go through short-lived signed URLs with an ownership check on the storage key; size-capped uploads.
-

This is the mechanism that keeps an app hardened *over time* instead of decaying one feature at a time. It is the single highest-leverage thing in this standard, and it is what makes repeated audits come back clean.

5.1 Why it exists

Across many full audits of a mature app, ~~700 findings resolved into~~ **15 recurring root-cause classes** re-instantiated in every new feature — not 700 unrelated bugs. The reason: cross-cutting invariants lived as **copy-pasted logic** rather than a shared primitive whose non-use fails the build, so every invariant silently regressed the moment a new surface was added. Patching instances never stopped the class; each returned in a new call-site.

5.2 The recurring classes and their seams

For each class: build **one shared primitive** ("the seam") every call-site must route through, then add a **static-analysis CI gate** that turns non-use into a build failure. Apply it **both ways** — retroactively (migrate every existing call-site until the gate is green, which by definition means no instance of the class is left) and preventively (block new ones).

#	Recurring class	The seam (one shared primitive)
1	Provider/client-trusted entitlement grants	A single grant chokepoint that verifies collected amount+currency vs booked, is monotonic on tier-rank, and is idempotent. Only an allowlist of files may write <code>tier / role / balance</code> .
2	Client-trusted idempotency keys	Namespace the client key under a server-derived scope against a unique constraint; never use the raw client header as a spend key.
3	Count-then-create TOCTOU on quotas/caps	One atomic <code>consumeQuota</code> (<code>INSERT ... WHERE used < cap</code> in a serializable tx); no non-transactional count → create.
4	Fail-open flags & test-mode	Unknown flag → <code>false</code> ; every flag key registered; every <code>*_TEST_MODE</code> flag enumerated in a prod boot guard.
5	Comms/tier-gate parity leaks	One gate applied on <i>every</i> read surface and one <code>assertCanSend</code> on <i>every</i> first-contact write.
6	Retrieval-exclusion parity	One <code>candidateBaseFilter</code> reused by every feed and search, so a banned/blocked/hidden entity appears in zero surfaces.
7	Realtime/session teardown on revoke	A central revoke/entitlement-change event drops sockets, ends calls, and de-indexes — entitlement is re-evaluated on change, not only at initiate.
8	Account-status authz	<code>assertAccountActive</code> in the JWT guard and every mint rail; status change revokes all sessions.
9	AI/paid-API denial-of-wallet	A cost guard reserves quota + a hard budget stop before the provider call; no vendor SDK imported outside it.
10	IDOR / entity-passthrough PII	Global ORM omit of secret columns + mandatory serialization DTOs + a storage-key ownership check.
11	Phantom permission slugs / dead wiring	Every permission slug, cron, and flag has a live referent or is explicitly marked dormant.
12	i18n drift	Key-parity + ICU-parity gates; no hardcoded user-facing strings.
13	Docs / help-center drift	Help content generated/verified from the same config that drives the app.
14	Modal a11y & color-only status	All modals use the shared accessible dialog shell; a lint rule bans raw modal primitives.
15	Input-normalization / validation bypass	Shared normalization + a ban on inline-object request bodies.

5.3 The ratchet: gates that can only grow

The prevention only works if the gates can never shrink:

- The class gates live as **static-analysis specs** (one `describe('... gate ...')` block per class) that scan the source and fail on regression. Many ratchet against a **shrink-only allowlist/baseline** of known not-yet-clean call-sites — you may remove entries as you clean them, **never add** new ones.
- A **meta-gate** counts those `describe` blocks and asserts the count never drops below a baseline. Deleting a gate (or gutting the file) to turn a red build green turns CI red instead. When you legitimately add a gate, bump the baseline in the same change — the only edit that file ever needs.

5.4 The standing rule (the ratchet on the ratchets)

Every finding — a known recurring class or a first-time novel issue — is closed by **ADDING** or **TIGHTENING** an enforced gate, never by a one-off patch. A gate is never quietly deleted to turn a red build green. The set of "can never happen again" classes only ever grows.

Supporting loop: a **known-bugs regression suite** (every past finding becomes a test that reproduces it and asserts it is blocked, so it can never silently reopen), a **continuous self-audit** on a schedule, and a **one-way ratchet** so each issue-class count can only decrease.

5.5 Lint-level architecture rules

Some seams are enforced at lint time in the clients too — for example a `no-restricted-syntax` rule that **bans the raw modal primitive** so every dialog uses the accessible shell (seam 14), and the a11y lint rulesets that fail the build on new violations (seams 12/14). Treat lint as another place to make an anti-pattern impossible, not just to enforce style.

MUST for any app holding personal data:

- **Data export** (GDPR/DPDP "right to access") — an async job producing a downloadable archive behind a **256-bit random token** (only its hash stored), every read path re-checking ownership, a short-lived signed URL that is never persisted, a size cap, a dedupe window, and no PII in logs. Sensitive documents (e.g. ID images) are **excluded** from the export.
- **Erasure** (right to deletion) — a cooling-off window, then **retention-aware**: *anonymize* (scrub PII, keep immutable finance/audit rows) when records must be retained or the account is under legal hold, else *hard-delete* with object-storage cleanup. A legal-hold erasure is parked, never silently dropped; erasure emits the revoke event that ends live sessions/calls.
- **Consent versioning** — an **append-only** consent ledger; every accept/withdraw is a new immutable row bound to the **server's** current policy version (never a client-supplied version), with IP/UA + a content hash.
- **Age-gating** — a hard minimum-age gate using leap-safe calendar arithmetic; under-age → block (+ ban/session revoke/audit where the app requires a minimum age legally).
- **Data-residency & minimization** — store only what the feature needs; keep a **durable vs. derived** split so only the primary store (Postgres + object storage) must be protected while caches/search are disposable and fail open/fall back. Keep a written **threat model** (Section 10.6) and a **privacy impact assessment**, updated whenever a trust boundary or data class changes.

MUST:

- **Additive-only, forward-only migrations.** No destructive DDL in a normal change; a code rollback must be safe without a DB downgrade. Migrations apply atomically on deploy as the release command.
- **Resilient write paths** — money/grant paths are idempotent and exactly-once (4.10); background work is queued and retried; a double-submit never double-charges.

- **Graceful degradation** — disposable dependencies (cache, search) **fail open or fall back** (e.g. search falls back to Postgres). Critical dependencies (primary DB, object storage) surface as `503`, not a crash.
- **Run ≥ 2 instances** with headroom for memory-hard hashing; health checks drive rolling deploys.
- **Performance budgets** — pagination is clamped and indexed; N+1 queries are avoided; the web bundle has a **first-load size budget enforced in CI** (12.3); images are variant-generated and served responsively; a **Lighthouse** budget (performance/best-practices/SEO) runs on the web app.
- **Load & DR** — a load-test harness (e.g. k6) with SLO thresholds, run on a schedule; a **disaster-recovery restore drill** run on a schedule against throwaway infra, verifying backup restore + row-count parity.

SHOULD: caching with explicit TTLs and invalidation; a circuit breaker in front of every external provider; RED (Rate/Errors/Duration) metrics on every route.

Standard: WCAG 2.2 Level AA, treated as a build requirement equal to types and tests — not a post-launch task. It is **CI-gated** (a11y lint rulesets fail the build on new violations; axe assertions run in the unit suites and e2e).

The **audit-first discipline** when building or changing any component:

1. **Audit first, change second.** Run the platform-appropriate accessibility checklist on the target file and output the report *before* touching anything. If a component has more than ~5 violations, report the full list and confirm before applying a large diff.
2. **Additions only.** Accessibility fixes add attributes; they **never** change rendering logic, navigation, API calls, state, or layout. If a fix would require any of those, flag it for review instead of silently applying it.
3. **One platform at a time.** Web/admin use HTML semantics + ARIA (`aria-label` , `role` , semantic elements); mobile uses React Native props (`accessibilityLabel` , `accessibilityRole` , `accessibilityViewIsModal`). Never mix them.
4. **Performance-safe.** All accessibility-info API calls go inside an effect with cleanup; complex labels are memoized; never compute a label inside render.
5. **New components run the platform checklist before they are marked done.**

Hard "never" list: `onPress` on a plain view without a button role; removing the focus outline on web without a replacement focus style; color as the **only** status indicator; a modal without the modal/ `aria-modal` treatment; announcing inside render; duplicate labels on parent and child.

MUST for any app shipping more than one locale (and structurally, even for one, so a second is cheap):

- **No hardcoded user-facing strings** — every string comes from the catalogue; a lint rule bans literals.
 - **One catalogue, all surfaces** — web (`{var}` placeholders), mobile (`{{var}}`), and backend read the same message set.
 - **Parity gates in CI** — every shipped locale has the **exact same key set** (missing *and* extra keys both fail), and **ICU parity** (placeholder names and plural/select branch keywords match the source locale).
 - **RTL correctness** where an RTL locale ships; locale-aware number/date/currency formatting (never bare `toLocaleString`); a locale/RTL registry as the single source of truth.
-

10.1 Structured, correlated logging

JSON logs in production (pretty in dev), **PII- and secret-redacted** (4.6). Every line carries `requestId` (the inbound `x-request-id` or a fresh uuid), `traceId` (when a span is live), and `userId` (once a guard resolves it), bound **pre-guard** into async-local storage so even guard-phase logs (like a 429) correlate; the response echoes the id.

10.2 Metrics

Prometheus-style metrics at a scrape endpoint that is **fail-closed in production** (403/404 unless an auth token is set). RED HTTP metrics + cache/rate-limit/business counters.

10.3 Tracing

OpenTelemetry only, exporting **only** when an OTLP endpoint is configured, else a safe no-op that still propagates trace ids. Keep the error tracker and the tracer from fighting over global instrumentation (initialize the error SDK so it does **not** patch HTTP or take over the OTel globals). Static gates keep tracing options consistent.

10.4 Error tracking

An error tracker (Sentry-class) across all platforms; on the backend capture **only 5xx**. It has a safe no-op mode when unconfigured. Incoming production errors are triaged and fixed, each fix shipped with a regression test that would have caught it.

10.5 Health & self-monitoring

Liveness/readiness probes that **do not disclose topology**. A self-monitoring layer: a system-health board, nightly reconciliation/data-integrity sweeps, and auto-healing of *safe* drift — never auto-healing by deleting data, weakening security, or disabling a safety control.

10.6 Security & operations docs

Keep these living documents in every repo and update them when a boundary/flow/data-class changes:

- **Threat model** (STRIDE-oriented: assets, trust boundaries, threats + the mitigation already in code, residual risks).
- **Privacy impact assessment** and a **data-residency & privacy** note.
- **Runbook** (how to operate) and a **DR runbook** (how to restore).
- A **bug-bounty / responsible-disclosure** policy.
- The generated **API contract** (OpenAPI), regenerated and stale-checked in CI.
- Living build docs: an architecture map (with ADRs for each significant decision), a build log, a fixes log, and a **pending-owner-actions** file for anything that genuinely needs the owner.

11.1 The production-safety split (non-negotiable)

Unit tests are pure logic and **PROD-SAFE**; end-to-end tests boot the real app and truncate the database, so they run **only** against a throwaway, ephemeral stack. A destructive suite must be structurally incapable of running against anything that could be production:

- The default `test` command runs **unit only**, so "run the tests" can never nuke a real database.
- E2E is opt-in, runs serially (it shares one database), and points at a **throwaway Docker stack on non-default ports**; inline env vars override any `.env` so it can't accidentally read production credentials.

11.2 What to test

- **Unit** — pure logic, no DB, no app boot; co-located with the code, dependencies stubbed.
- **Integration** — one test per **declared blast-radius touchpoint** (Section 14), proving the touched module still behaves with the new feature present. Non-deferrable for Medium/High-risk changes.
- **E2E** — boots the real app against the throwaway stack and exercises HTTP/flows end to end.
- **Safeguard-proving tests** — tests that prove the *security controls* work: N-parallel idempotency (one effect), post-ban socket/call count is zero, a banned profile appears in zero surfaces, no secret field names in any response, no PII in logs, rate-limit returns the structured 429. A control without a test that it fires is not trusted.
- **Client** — component tests assert behavior *and accessibility* (axe); e2e (Playwright web / Detox mobile / a Maestro smoke that catches crash-on-launch).

11.3 Coverage gates

Enforce coverage as a **floor that blocks a real regression without flaking on small swings**, and **ratchet it up as coverage grows — never lower it below the real numbers**. Put **hard higher floors on the deeply-tested critical surfaces** (billing, core product, onboarding) so they can't erode to average. Backend and web carry numeric gates as the baseline; extend to admin/mobile as their suites mature.

11.4 Test online, not only on localhost

Test web/admin against deployed preview/staging URLs and mobile on real simulators pointed at deployed staging. In any environment, use what is actually available, **record what could not be verified, and never claim a test ran where it couldn't**.

11.5 Mobile testing discipline

Mobile has two hard-won CI rules: run the suite **serially with a forced exit** (a platform-specific open handle keeps the Linux runner alive after passing tests — safe, teardown-only), and **fail on changed snapshots under `--ci`** so a component change must commit its updated snapshot in the same PR. Never leave fake timers pending across files (it hangs the runner).

This is the concrete answer to "the checks that must be done before any PR merges." **Green CI is the merge bar**. Everything below runs automatically; humans do not gate on anything a machine can gate on.

12.1 The pipeline at a glance

Workflow	Trigger	What it enforces
Backend CI	push/PR on backend paths	lint · typecheck · unit + coverage gate · e2e (real DB/Redis/search services) · OpenAPI staleness · generated-docs drift · entrypoint smoke

Workflow	Trigger	What it enforces
Web CI	push/PR on web paths	i18n key + ICU parity · lint (incl. a11y) · typecheck · unit (incl. axe) + coverage · build · bundle-size budget · Playwright e2e · Lighthouse (non-blocking) + PWA installability
Admin CI	push/PR on admin paths	i18n parity · lint (incl. a11y) · typecheck · unit (incl. axe) · build
Admin-MCP CI	push/PR on MCP paths	typecheck · coverage-drift test (admin surface fully reachable) · build
Drift gates	push/PR + daily schedule	prevention-standard files present · self-healing hook wired end-to-end · recurring-class architecture gates + monotonicity meta-gate
Mobile (Android/iOS)	push/PR on mobile paths	typecheck · lint · jest unit
Mobile smoke / Detox / bundle-verify	schedule + scoped PR	real-simulator launch, transactional flows, embedded-bundle (crash-on-launch) verification
Security scan	PR (blocking secret scan) + weekly	gitleaks on every PR ; weekly dependency audit, filesystem CVE scan, SAST
CodeQL	weekly	SAST (where code-scanning is available)
Deploy staging / production	push to default (staging) · manual (prod)	build image · migrate · zero-downtime machine guard
DR restore / load test	weekly schedule	backup restore parity · SLO thresholds

12.2 The backend pipeline

Ordered, and each step is a gate:

1. `install --frozen-lockfile` → generate the ORM client.
2. **lint** → **typecheck**.
3. **Generated-docs drift** — regenerate stats/data-model/API index and **fail on any** `git diff` (so generated docs can never go stale). A **self-healing pre-commit hook** regenerates and stages these on every commit so authors don't have to remember; CI is the backstop for `--no-verify`.
4. **Unit tests + coverage gate**.
5. **E2E** against real Postgres/Redis/search service containers.
6. **OpenAPI staleness gate** — re-export the contract and **fail if it differs** from the committed file, so the API contract (and everything generated from it, including the typed client and the Admin MCP's reach) is always current.
7. **Production-entrypoint smoke** — build the container artifact and assert the compiled entrypoint exists at the exact path the container's `CMD` runs, and actually boots. Guards the backend build-boundary trap (2.5).

12.3 The web pipeline

i18n parity gates → a11y-inclusive lint → typecheck → unit (with axe) + coverage → build → **bundle-size budget** (fails if shared first-load JS exceeds the baseline by more than a small margin) → **Playwright e2e**

run **fully offline/mock**ed across desktop + mobile viewports. Lighthouse + PWA-installability run on push (warn-level, not a PR blocker).

12.4 Drift gates & the prevention system in CI

A dedicated workflow (also on a **daily schedule** — continuous self-audit) asserts the prevention system stays intact: the standard docs exist and are non-empty; the self-healing docs hook is installed, wired, and passes a live scratch-commit smoke test; and it runs the **recurring-class architecture gates** plus the **gate-monotonicity meta-gate** (Section 5).

12.5 Security scanning

- **Secret scanning on every PR** (gitleaks-class), full git history, **blocking**. An allowlist covers only known CI test values; first-party source stays fully scanned.
- **Weekly (or on-demand)**: dependency vulnerability audit, filesystem CVE scan, and SAST (CodeQL-class). These are scheduled rather than per-PR where a paid code-scanning tier isn't available — document that choice.

12.6 Git hooks & required checks

- A **pre-commit hook**, installed by the repo's `prepare` script on every install via a delegating shim (works across worktrees), that **self-heals generated docs** and stages only what changed. It fails open when its runtime is missing; CI is the enforcement backstop.
- **Required status checks** on the default branch include at least the backend e2e gate and the admin-MCP coverage gate; pair each required check with a tiny **"required shim"** workflow that reports the same check name on PRs that don't touch those paths, so a required check never deadlocks an unrelated PR.
- **Branch protection**: never commit to the default branch directly; branch → PR → green CI → **squash-merge** (one commit per PR, the PR number in the subject).

12.7 Dependency hygiene

A dependency bot (weekly, grouped by ecosystem, PR-limited) for npm, CI actions, and Docker base images. Pin security-sensitive transitive dependencies with overrides. Keep one lockfile; frozen-lockfile installs everywhere.

12.8 Conventions & attribution

- **Conventional Commits** for commit subjects and PR titles (`type(scope): imperative summary`).
- The PR template requires: summary, why (+ linked issue), a **test-plan checklist** (unit / integration / e2e / manual web / manual mobile / PWA offline / i18n / no new lint or type errors / **no new security findings**), screenshots for UI, **risk**, and a **rollback plan**.
- **No AI/authorship attribution anywhere** — not in code, comments, docs, commit messages, PR titles/bodies, config author fields, or generated banners. If a tool inserts such a tag automatically, remove it.

MUST:

- **Containerized, reproducible builds** — multi-stage Docker on a pinned base; production dependencies only in the runtime image; the ORM client generated at build.
- **Per-app isolation** — a uniquely-named container project (its own container names, networks, volumes, ports) so apps never collide on a shared machine.
- **Migrations run as the release command** and apply atomically on deploy; they are additive/forward-only so a rollback is safe without a DB downgrade.
- **Secrets are platform secrets**, never in the repo; a strict-secrets switch hard-fails a production boot on any dev-default secret.
- **Zero-downtime deploys** — run ≥ 2 serviceable instances; a deploy guard refuses to proceed below the safe machine count; health checks drive the rollout.
- **Private networking** for internal services (search, cache) — not publicly reachable.
- **Staging that mirrors production** and can scale to zero when idle; production deploys are **manual and guarded** (type-to-confirm), staging deploys on merge to the default branch.
- **Mobile release**: managed build/submit/OTA (EAS-class) with an **OTA guard that refuses to push a native/config change over-the-air** (native changes must go through a store build), and an **embedded-bundle verifier** that proves the shipped binary contains a non-empty JS bundle before store submission (catches crash-on-launch).
- Local dev runs the **whole service topology in Docker** (DB, cache, search, object storage, mail capture, and any realtime infra) via one compose file, on non-default ports so it never collides with other projects.

MANDATORY before writing code for any feature/module/change — including small-looking ones (small changes are exactly the ones that slip through). It sits between spec-writing and building, is executed by the builder, and is enforced by CI. It is **never an owner review task**.

Before coding, produce and save a **Blast Radius Declaration** to the repo's `BLAST_RADIUS_LOG.md` :

1. **What it does** — one plain-language paragraph.
2. **Blast Radius Declaration** — every existing module / file / DB table+relation / shared service / external integration the change will **read from**, **write to**, or **depend on the current behavior of** (including things it doesn't modify but assumes work a certain way). *Be exhaustive — a touchpoint you didn't declare is a touchpoint you didn't test.*
3. **Risk tier** — **Low** (0–2 touchpoints) · **Medium** (3–4) · **High** (5+, or anything touching identity/auth, billing/money-ledger/accounting, or verification/trust-&-safety, regardless of count). A change touching a known repeat-offender module is at least Medium.
4. **Required integration tests** — one mapped to **each** declared touchpoint, written down **before** coding.
5. **What changes in existing behavior** — explicit. *"Nothing changes for existing functionality" is a required line to write when true* — a claim to be proven by the touchpoint tests, never an unstated assumption.

Hard rule: for Medium/High changes, all declared integration tests **must exist and pass** before the change is done — non-deferrable.

Parallel-work overlap rule: when multiple agents/branches build in one sprint, each produces its own declaration; a lead collects them **before any merge** and resolves any **overlapping touchpoint** (same module/file/table declared by more than one) — ordering, whether the other must adapt, whether a joint

integration test is needed — logging the overlap and resolution. Merging before the overlap check is a gate violation.

The log is a persistent audit trail; its **repeat-offender** section (modules that recur as touchpoints across many features) is re-read at each audit cadence, and those modules get dedicated integration coverage independent of any single feature.

15.1 Sprints — always shippable

Work an ordered sprint plan; each sprint ends **working, tested, committed, pushed, and deployed as far as possible**. A representative order:

0. **Isolated foundation & deployed pipeline** — new repo/Docker project; CI/CD; hosting/storage/email/error-tracking wired; the standard vendored into the repo; the **architecture-gate scaffold + meta-gate + drift-gates workflow armed**; a "hello world" flowing commit-to-production, tested online.
1. Identity & Access → 2. Account/Profile + Onboarding → 3. Billing & Accounting → 4. Core Product → 5. Messaging → 6. Notifications, Search, Trust & Safety → 7. Admin & Owner Console + Audit + Support (help-center-as-code) → 8. Settings, Reporting, Data/Privacy, Integrations → 9. Observability, self-monitoring & hardening + a cross-cutting sweep → 10. Multiplatform polish, marketing surface, release readiness.

Apply the cross-cutting requirements and Definition-of-Done checklists **as each module is built**, not as a later pass. **Never stop at a skeleton**.

15.2 The Feature Intake Interview (owner-requested product work)

Do **not** build owner-requested features from a one-line description. First ask a structured round of plain-language clarifying questions — who it's for; what the user sees step by step; the awkward cases ("what if they cancel halfway?"); wording and tone; pricing/tier placement; what staff and owner see; what "done" looks like — as concrete options **with a recommendation**, and mockups where a picture decides faster. Close with a two-minute plain-language mini-spec for the owner's go-ahead; that mini-spec feeds the Blast Radius Gate. After building, run a **UX acceptance pass** — demonstrate the working feature visually and iterate on feedback.

This applies to **owner-requested product work only**. Technical work — fixes, refactors, hardening, infrastructure — proceeds autonomously with zero questions.

15.3 Autonomous, parallel execution

Build autonomously and safely, maximizing speed and quality with parallelism: dispatch parallel subagents per module or platform against the shared backend/typed-client/types, with a lead integrator owning the contracts and continuously integrating. Use **git worktrees** so parallel builds never collide, created **inside the app's own folder** and cleaned up when done. Give finance/accounting, identity/access, and security/privacy the highest scrutiny. Don't over-fragment trivial work. Don't economize on tokens or time — being exhaustive and finishing fast both win.

15.4 Computer safety & isolation (highest priority)

Overrides convenience, always:

- Work **only inside this app's own folder**. Never touch other apps' code, worktrees, services, containers, or unrelated files.
- Never change crucial system settings, system files, or global config; never run privileged machine changes. Prefer Docker and project-local over global installs.
- **State the impact before anything more than very-low-risk to the machine**, and record it for the owner rather than doing it.
- Each app is fully isolated (own repo, own uniquely-named Docker project, own cloud resources).

15.5 Never halt on a human-only blocker

When something genuinely needs the owner (creating an account, real payment credentials, DNS, an app-store submission, rotating a secret), **do not stop**: stub or mock it cleanly, record it in

`PENDING_OWNER_ACTIONS.md` (what to do, where, the exact values, what is stubbed, how to unstub), and keep building everything else. Make unstubbing a config/secret change, not a rewrite.

15.6 The standard travels with the repo

At Sprint 0, **vendor this standard into the app repo** (`docs/standards/` + short `CLAUDE.md` pointers), with a CI check that the guardrail docs exist — so cloud/remote sessions that clone the repo fresh obey the same rules. Keep `CLAUDE.md` short: it points at this standard, names the Blast Radius Gate as mandatory, and points at `BLAST_RADIUS_LOG.md` — it does not restate them.

15.7 The help center is part of the product

If the app has any user-facing help/FAQ, it is **help-center-as-code**: the canonical content lives in the repo and is served to every client. Whenever you build, change, or remove a user-facing feature, flow, plan, price, limit, or policy, **update the help content in the same change** — "the help center still matches the app" is part of the Definition of Done, mechanically asserted where possible (a test that ties advertised tiers/limits to the runtime config). Help content is **shared, general guidance only** — never customer-specific or financially sensitive data. Voice: warm, plain, to the point, professional.

Every app serves the owner and staff as first-class users:

- **Drill-down dashboards** — every tile/number is clickable through to the underlying records; the owner can see and control everything.
- **Safe feature toggles** — a runtime on/off system that **never deletes data**; most new/risky features ship **dormant behind a default-off flag** and are flipped on later (an unregistered flag must **fail closed**). This is how monetization/risky work lands on the default branch without changing production behavior.
- **"View as"** — read-only impersonation for support (4.3).
- **The Admin Control MCP** — a Model Context Protocol server that makes the **entire** admin/staff surface machine-operable. It works by a **generic, contract-driven proxy** (list operations from the committed OpenAPI contract; invoke any of them by method+path) scoped to the `/staff` and `/admin` prefixes, plus a thin layer of curated tools for high-traffic flows. Because it reads the always-fresh OpenAPI contract, **any new admin endpoint becomes controllable with no MCP code change** — a coverage-drift test fails if a `/staff` or `/admin` route falls outside the configured prefixes. Its safety model: a default read-only operating profile, confirm/dry-run (with an impact preview) for destructive operations,

PII redaction by default, gated sensitive tools, a large-action threshold, scoped service roles (never run it as owner), and every mutation flows through the same backend audit log. **Definition of Done:** when you add/change/remove any `/staff` or `/admin` endpoint, in the same change regenerate the contract, add any new prefix, add a destructive-classifier rule where needed, and keep the coverage test green.

A change is **DONE** only when all of the following hold. Hard requirements (**!**) block "done" unconditionally and are enforced by machine wherever possible.

Blast radius

- ☐ **!** Blast Radius Declaration completed and saved (what it does · touchpoints · risk tier · integration tests · behavior-change line).
- ☐ **!** Risk tier assigned; for parallel sprints, the overlap check performed and any overlap resolved + logged.
- ☐ **!** All required integration tests exist and pass — one per declared touchpoint (Medium/High: non-deferrable).

Correctness & tests

- ☐ **!** Unit tests for the new code, and the **full existing suite passes** (not just new tests) on the isolated stack — never against production data.
- ☐ **!** Typecheck + lint clean across all affected apps and packages.
- ☐ Safeguard-proving tests exist for any security/money/safety control the change adds or touches.
- ☐ For any change in a recurring class: routed through the shared **seam**, and the class's **enforcement gate is green** (no old-pattern call-sites left).

Safety & back-compat

- ☐ **!** Migrations are **additive only** (no destructive DDL).
- ☐ Behavior changes ship behind a **new default-off flag** (fails closed).
- ☐ Safety/money/entitlement/authorization gates **fail closed**.

Product completeness (no skeletons)

- ☐ **!** No skeleton/stub in a shipped path; dormant-by-design surfaces are explicitly annotated, not silently inert.
- ☐ Complete lifecycle + every state handled + robust lists/forms for every entity touched.
- ☐ All three audiences (user, staff/admin, owner) served for the feature.
- ☐ Audited, notified, measured — meaningful actions logged, correct notifications, analytics emitted.

Cross-cutting

- ☐ **!** **Help center updated in the same change** if any user-facing feature/flow/plan/price/limit/policy changed.
- ☐ **!** **Admin surface (and the Admin MCP contract) regenerated/green** if any admin/staff endpoint changed.
- ☐ **!** **Accessibility** platform checklist run for new/changed components (WCAG 2.2 AA).
- ☐ **!** **i18n:** no hardcoded user-facing strings; all shipped locales at parity for touched keys.

Hygiene

- ☐ **!** No AI/authorship attribution anywhere (code, comments, commits, PR body).
- ☐ Docs describing the changed behavior updated in the same change (no drift).
- ☐ PR filled out (summary, why, test plan, risk, rollback); green CI; squash-merge.

The single test still governs: *could a lean team run a real business on this the day it ships, without a backlog blocking operations?*

Do these before the first feature. This is what makes an app hardened *from day one* instead of retrofitted.

Repository & isolation

- ☐ New repo; new uniquely-named Docker project (own container names/networks/volumes/ports).
- ☐ Monorepo layout (3.2); one lockfile; pinned Node + package-manager versions.
- ☐ This standard vendored into `docs/standards/`; a short `CLAUDE.md` pointing at it + the Blast Radius Gate; `BLAST_RADIUS_LOG.md`, `DEFINITION_OF_DONE.md`, `PENDING_OWNER_ACTIONS.md` created.

Stack

- ☐ Backend (NestJS + Prisma + Postgres), web + admin (Next.js PWA), mobile (Expo) scaffolded against **one versioned API** and a **generated typed client**; shared packages extracted (2.5).
- ☐ Local service topology in one Docker compose file on non-default ports.
- ☐ Every external channel behind a **provider interface with a dev/log provider** (2.4).

Security baseline (Section 4)

- ☐ Zod env schema + boot validation + strict-secrets switch; secret scanning on PRs; `.env.example`.
- ☐ Auth (short-lived JWT, rotating refresh with reuse detection, argon2id, OTP, sessions, account-status gate, staff MFA).
- ☐ Global rate-limit guard + login lockout; helmet headers + CSP/HSTS; CORS allow-list; API docs gated out of prod.
- ☐ Append-only hash-chained audit log; RBAC guard with seed-backed permission slugs; read-only impersonation.
- ☐ Global secret-column omit + serialization DTOs; validation pipe (whitelist + reject-unknown); log redaction.
- ☐ Field-level encryption + blind indexes for the most sensitive PII; privacy module (export/erasure/consent/age-gate).

Prevention system (Section 5)

- ☐ Architecture-gate spec file seeded with the recurring bug classes; the **monotonicity meta-gate**; a shrink-only allowlist per gate.
- ☐ Drift-gates workflow armed (present-files check + hook-wiring check + the class gates), on push/PR **and a daily schedule**.
- ☐ Self-healing pre-commit hook installed via the `prepare` script; known-bugs regression suite started.

CI/CD & delivery (Section 12)

- ☐ Per-app CI (lint · typecheck · unit + coverage · e2e/build) with the production-safety test split.
- ☐ OpenAPI staleness gate + endpoint smoke; i18n parity gates; a11y lint + axe; bundle budget; Playwright.
- ☐ Blocking secret scan on PRs; scheduled dependency/CVE/SAST scans; dependency bot.
- ☐ Branch protection (no direct pushes; PR → green CI → squash-merge); required checks + required shims; PR template.
- ☐ Staging on merge; guarded manual production deploy; migrations as the release command; scheduled DR restore + load test.

Observability (Section 10)

- ☐ Correlated structured logging (redacted); metrics endpoint (fail-closed in prod); OTel tracing; error tracker; health probes; threat model + runbook + DR runbook started.

Each of these is a "not done" condition, most of them machine-enforced:

- **The skeleton** — shipping "it renders" as if it were "it works."
- **The one-third feature** — building for the end user but not for staff and owner.
- **The spot patch** — fixing a bug-class instance without adding/tightening the gate that makes the class extinct.
- **The deleted gate** — removing or gutting an enforcement gate to turn a red build green.
- **The prod-database test run** — pointing a destructive suite at anything that could be production.
- **The trusted client** — believing the client's tier, price, role, payment reference, idempotency key, or self-set country/IP.
- **The fail-open gate** — an authz/money/safety/entitlement decision that passes on a missing or unknown signal.
- **The half-revoked entitlement** — a ban/suspend/block/logout that doesn't drop live sockets, end calls, and de-index.
- **The raw secret in a response or log** — returning a Prisma model with credential columns, or logging PII.
- **The technical owner task** — handing the non-technical owner code to review or commands to run.
- **The unasked product guess** — building an owner-requested feature from one line without the intake interview.
- **The stranded standard** — an app whose rules live only in someone's head or on one machine, not vendored in the repo and enforced by CI.
- **AI/authorship attribution** — anywhere in the codebase, docs, commits, or metadata.

This standard is a living document. Extend it the same way the code is extended: every new lesson becomes a new rule and, wherever possible, a new machine-enforced gate — so the bar only ever rises.